A Fast Dynamic Graph Convolutional Network and CNN Parallel Network for Hyperspectral Image Classification

Quanwei Liu, Yanni Dong, *Senior Member, IEEE*, Yuxiang Zhang, *Member, IEEE*, and Hui Luo

***Abstract*— Deep learning has achieved impressive results on hyperspectral image (HSI) classification. Among them, both convolutional neural networks (CNNs) and graph neural networks (GNNs) have great potential for HSI classification. Supervised CNNs can efficiently extract hierarchical spatial–spectral features of HSIs, but these methods face the problem of high time complexity as the number of network layers increases. Semisupervised GNNs can rapidly capture the structural information of HSIs, while they cannot be well extended to HSI applications because of the process of adjacency matrix consuming large amount of memory resources. In this article, we propose a fast dynamic graph convolutional network and CNN (FDGC) parallel network for HSI classification. We first obtain two classification features by flattening and pooling operations on the results of the convolution layers, which fully exploits the spatial–spectral information contained in the hyperspectral data cube. Then, a dynamic graph convolution module is applied to extract the intrinsic structural information of each patch. Finally, we can obtain the HSI classification results based on these spatial, spectral, and structural features. By using three branches, FDGC can parallelly process multiple features of HSI in a supervised learning manner. In addition, regularization techniques, such as DropBlock and label smoothing, are applied to further improve the generalization capability of the model. Experimental results on three datasets show that our proposed algorithm is comparable with the state-of-the-art supervised learning models in terms of accuracy while also significantly outperforming in terms of training and inference time.**

***Index Terms*— Convolutional neural networks (CNNs), dynamic graph convolutional networks (dynamic GCNs), fast parallel network, hyperspectral image (HSI) classification.**

I. Introduction

HYPERSPECTRAL imaging technology is capable of providing detailed spectral information by sampling hundreds of narrow continuous spectral bands from the visible region (0.4–0.7 μm) to the short-wave infrared (SWIR) region (0.7–2.4 μm) [1]. Applications based on hyperspectral images (HSIs) are widely developed, such as agricultural assessment, environmental inspection, mineral exploration, and military reconnaissance [2]–[5]. However, the high-dimensional property of HSIs makes the topological relationship between high-dimensional units change essentially, such as the data distribution density becoming sparse and the spatial centroid being outside the hypersphere [6]. If the usual data processing methods based on RGB images are applied directly to high-dimensional HSI, it results in poor performance [7]. Therefore, a series of processing algorithms has been dedicated for HSIs target detection, unmixing, classification, and so on. Among them, HSI classification has attracted more and more attention [8]–[11].

HSI classification usually consists of two parts: feature extraction and classification. Extracting discriminative features can significantly improve the final classification accuracy [7]. The earlier feature extraction methods are designed manually with the help of machine learning methods for HSI classification. For example, principal component analysis (PCA) [12], random forest (RF) [13], and support vector machine (SVM) [14] based on spectral information were proposed. Morphological profiles (MPs) [15], Markov random field [16], and 2-D Gabor filters [17] based on the spatial information were proposed. Nevertheless, it is difficult for the above methods to simultaneously utilize the rich spatial and spectral information contained in HSI. Some joint spatial–spectral methods can make better use of the properties of HSI data cubes, namely, 3-D Gabor wavelets [18], [19] and morphological information methods. In addition, manifold learning [20] and graph representation learning [21] can further construct the structural information of hyperspectral data. Although these methods have achieved remarkable success in HSI classification, machine learning faces the problems of manual feature design, shallow feature extraction, and so on [22].

In contrast, deep learning (DL) can automatically extract high-level abstract features [23], [24]. After the success on RGB images, DL models, typified by convolutional neural networks (CNNs), are applied to HSI classification. The stacking CNN-based model fusion algorithms have obtained good results in HSI classification. Stacking is a model fusion method that fuses the predicted results of several single models in one model with the goal of reducing the generalization error of a single model [25]. In CNNs, it is usually represented

Manuscript received March 2, 2022; revised May 3, 2022 and May 15, 2022; accepted May 28, 2022. Date of publication May 30, 2022; date of current version June 10, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62171417, Grant 62071438, and Grant 41801285; and in part by the Open Research Project of the Hubei Key Laboratory of Intelligent Geo-Information Processing under Grant KLIGIP-2018A01. *(Corresponding author: Yanni Dong.)*

Quanwei Liu, Yanni Dong, and Yuxiang Zhang are with the Hubei Subsurface Multi-Scale Imaging Key Laboratory, Institute of Geophysics and Geomatics, China University of Geosciences, Wuhan 430074, China (e-mail: liuquanwei@cug.edu.cn; dongyanni@cug.edu.cn; zhangyx@cug.edu.cn).

Hui Luo is with the School of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: huiluo@cug.edu.cn).

Digital Object Identifier 10.1109/TGRS.2022.3179419

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

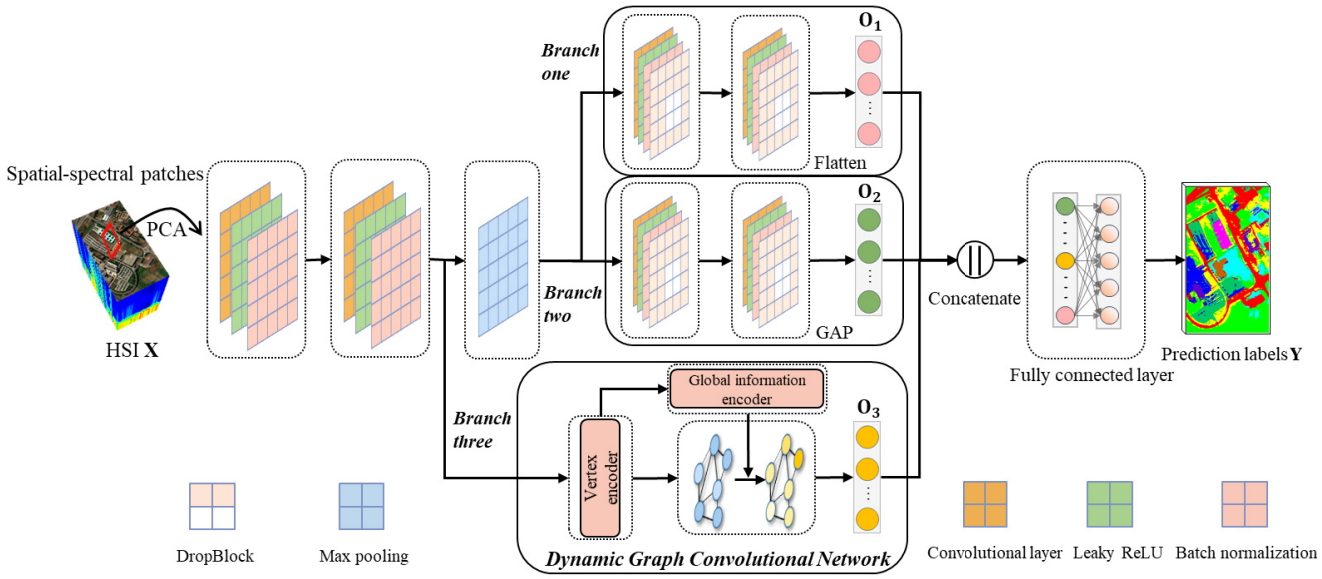


Fig. 1. Architecture of the proposed FDGC, where **X** denotes the input HSI. Flatten means to flatten the feature map into a column vector. GAP is a global average pooling operation. The operator || is concatenating features along the spectral dimension, and **Y** denotes the predicted result.

as a combination of multibranch convolutional networks. For instance, Lee and Kwon [25] were the first to introduce the inception module into HSI classification, using a combination of 1×1 and 3×3 convolutional layers to extract spatial and spectral features of HSIs, respectively. Spectral–spatial unified networks [26] proposed a band grouping-based long short-term memory model to extract spectral feature and a multiscale CNN to extract spatial feature, which together constitutes an end-to-end architecture for the HSI classification. To extract more robust and effective features from HSI, Gao *et al.* [27] proposed an HSI classification framework with four branches, which can simultaneously learn several types of robust high-level deep features. Moreover, inspired by the visual attention mechanism of human visual perception, CNNs combined with the attention mechanism are applied in the stacking framework. TAP-Net [28] proposed a novel triple-attention module that consists of channelwise, spectralwise, and spatialwise attention. In [29], the multiway attention modules in spectral and spatial branches were built, and the multiscale spatial block in the spatial branch was designed, which can efficiently extract vital informative spectral–spatial features and suppress useless features from abundant spectral–spatial information. However, the nature of CNN local connectivity and weight sharing results in the need for more layers if a larger perceptual field is desired. Deeper CNNs require a lot of time for training [30].

Recently, graph neural networks (GNNs) have been developed due to its capability of fast feature extraction for the prevalent non-Euclidean data [31]. GNNs can construct graph convolution layers through the adjacency matrix to capture global features in one layer. This makes it possible to fully extract the information of HSI with excellent performance by a few graph convolution layers. To fully utilize the information of HSI, Qin *et al.* [32] first proposed a semisupervised graph convolutional network (GCN), which combines the intrinsic information of labeled and unlabeled samples to alleviate the problem of insufficient samples. MDGCN [33] utilized superpixels instead of pixels as nodes of the graph to greatly reduce the memory consumption due to the construction of adjacency matrix. Nevertheless, multiscale superpixels inevitably misclassify some pixels, resulting in smooth edge information and limited classification accuracy. Liu *et al.* [34] proposed a two-branch semisupervised classification method to simultaneously extract features on both superpixel and pixel levels. However, HSIs are so large that even with the superpixel segmentation, the semisupervised GNN still takes up a lot of video memory, which limits its applicability [35].

As the problems of the above DL-based models, namely, high time complexity of multilayer CNN and high space complexity of semisupervised GNN, we propose a fast dynamic graph convolutional network and CNN (FDGC) parallel network, as shown in Fig. 1. Instead of utilizing a predefined fixed graph for convolution, the FDGC uses an attention-based dynamic graph convolutional network (dynamic GCN) to extract the topological structure information of HSIs, which can be used as a plug-and-play replacement for normal convolutional layers. This enhances the flexibility of the traditional GCN and breaks the limitation of the CNN local field of perception, which will in turn make the feature embeddings more accurate and feature extractions faster. The processes of graph updating and feature embedding work in concert to produce a faithful graph structure and encouraging classification results for each patch. CNN has its unique advantage in extracting local information. Therefore, to take the multiscale cues into consideration, two CNN branches are added to the network structure. Different from commonly used CNN layers which stack only the same branches, the FDGC extracts spectral–spatial features with flattening and pooling operations, by which the comprehensive contextual information from different levels can be incorporated. Moreover, due to the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

large number of pixels brought by the HSIs, the computational complexity of network training can be extremely high. To mitigate this problem, we use a parallel fusion mechanism to train the network in a supervised learning manner, which helps the network to scale to larger graphs. In addition, since DL model integration can lead to severe overfitting, we use DropBlock and label smoothing regularization methods to enhance the generalization ability of the model. The main contributions of this article are given as follows.

1) To address the high time complexity of deep convolutional networks, we propose a hybrid DL framework for HSI classification by integrating dynamic GCN and CNN, which uses three branches to process HSI in parallel to enable fast information extraction.
2) To address the high spatial complexity of semisupervised classification GNN, we propose a novel dynamic GCN module that can extend GNNs to large graphs by adaptively capturing topological structure information in a supervised learning manner.
3) To alleviate the overfitting problem caused by the small sample problem in HSI classification, we adopt scheduled DropBlock to learn more generalizable features. To mitigate the interference of mixed pixels for labels, we use label smoothing to learn a more general label representation.
4) Experiments on three benchmark datasets show that the proposed framework can obtain competitive results at a faster speed compared with seven other state-of-the-art methods.

The remainder of this article is organized as follows. Section II reviews the related work about GNN and regularization. In Section III, the proposed FDGC method is described in detail. Section IV elaborates the experimental results obtained on three HSI classification benchmark datasets systematically. Finally, the conclusion is given in Section V.

II. Related Work

Several types of prominent works have been leveraged for deep feature learning. In this section, we recall some preliminaries of GNNs and regularization methods.

A. Graph Neural Networks

Graph data are commonly found in the real world, but the development of DL based on graph data is relatively slow due to the sequence disorder and dimensional variability of graph data [36], [37]. Bruna *et al.* [30] first proposed the concept of graph convolution based on traditional CNNs, which enables the expansion of DL from the Euclidean domain to non-Euclidean domain. Since then, graph-based DL algorithms have flourished along both the spatial domain and the spectral domain. As the understanding of GNNs deepened, Chen *et al.* [38] investigated the relationship between spatial- and spectral-based techniques, which exposed the connections with each domain.

Spectral approaches implement convolution operations on topological graphs with the help of graph theory. Unfortunately, the computational complexity of the graph Fourier transform, which must be used, is $O(n^2)$. Therefore, ChebNet was proposed by Defferrard *et al.* [39], which defines the Chebyshev polynomial of the diagonal matrix of the feature vector as a filter, to reduce the computational complexity. Moreover, first-order ChebNet [40] was proposed to further reduce the computational complexity of GCNs, which is comparable to the top semisupervised methods in terms of computational efficiency and accuracy. Subsequently, a large number of variants of GCNs have been developed based on this [41], [42].

Spatial approaches [43], [44] define the convolution operation directly on the connectivity of each node, essentially by continuously aggregating the neighbor information of nodes. Duvenaud *et al.* [43] proposed a model where all nodes in a neighborhood share the same kernel weights. GraphSage [44] implemented inductive training and testing by aggregating the information of neighboring nodes in a sampling and aggregation manner, which allows graph convolution to be easily extended to large graphs.

B. Regularization

Large capacity deep neural networks (DNNs) have achieved exceptional performance in many tasks. The network tends to memorize the samples, which yields poor generalization for data outside of the training data distribution. When training the DL models with limited labeled training data, it is easier to produce large gaps in training and testing performance. This phenomenon is also known as the overfitting problem [45]. To address this issue, many regularization strategies have been employed during training, making the model more robust against invariant samples and thus improving the generalization of the model [46], [47].

Dropout [48] is to temporarily drop neurons from the network with a certain probability during training, which can be considered as a practical bagging method for integrating a large number of DNNs. Ghiasi *et al.* [49] proposed a structured Dropout: DropBlock that removes consecutive units and forces the remaining units to learn more semantic information to enhance the model generalization ability. SpatialDropout [50] deactivated the whole channel of the feature layer in order to avoid the overall change of semantic information. FractalNet [51] proposed a regularization strategy for random deactivation of multibranch structures, which randomly removes branches in networks to reduce overfitting.

Direct training of hard labels often results in overconfident models. Boosting labels can efficiently alleviate the overfitting problem and improve the accuracy and robustness of DNNs. For example, bootstrapping [52] improved the robustness of models by smoothing hard labels in two ways, namely, Bootsoft and Boothard. Inception v3 [53] proposed a label smoothing strategy combined with uniformly distributed updated label vectors to avoid overconfidence in the correct labels. Xie *et al.* [54] presented DisturbLabel, which randomly replaces a part of labels as incorrect values in a mini-batch. Li *et al.* [55] used two networks to embed images and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

labels into a latent space separately and learn the relationship between them by deep distance measure for correcting the network.

III. Method

Concerning hyperspectral data, we denote it as $\mathbf{X} \in \mathbb{R}^{H\times W\times B}$ and $\mathbf{Y} = \{y_1, y_2, \ldots, y_c\}_{c=1}^{C} \in \mathbb{R}^{H\times W}$ is ground truth. H, W, B, and C denote the length, width, height, and the total number of categories of the HSI, respectively. The HSI classification determines the category to which each pixel belongs by a classification model. In this work, we propose the FDGC with three branches, which mainly consists of graph convolutional layers and convolutional layers. In the following, we will describe the framework in detail.

A. Dynamic GCN

1) Vertex Encoder: As mentioned before, GCNs can process the information of aggregated neighbor nodes to achieve feature extraction. However, due to the incompatibility of data representation between different network architectures, it is not straightforward to integrate GCNs and CNNs directly. The feature map $\mathbf{h} \in \mathbb{R}^{H\times W\times D}$ processed by CNN first goes through the vertex encoder module to obtain a set of vertex feature, each of which describes the contents related to a specific label from the input feature map. As shown in Fig. 2, the category-specific activation maps $\mathbf{M} = \{\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_c\}_{c=1}^{C} \in \mathbb{R}^{(H\times W)\times C}$, first calculated by vertex encoder, are used to convert the transformed input feature map $\mathbf{h}$ into the sequence representations $\mathbf{V} = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_c\}_{c=1}^{C} \in \mathbb{R}^{C\times D}$. This can be expressed as

$$\mathbf{v}_c = \mathbf{m}_c^T \mathbf{h}' = \sum_{i=1}^{H}\sum_{j=1}^{W} m_{i,j}^c \mathbf{h}_{i,j} \tag{1}$$

where $\mathbf{m}_c^T$ denotes the transposed weight of the (c)th activation map and $\mathbf{h}' \in \mathbb{R}^{(H\times W)\times D}$ denotes the feature vector after matrix deformation. Specifically, the generated vertex information can selectively aggregate class-specific relevant features and use them for subsequent processing.

2) Graph Convolutional Layer: With vertex representations $\mathbf{V}$ obtained in Section III-A1, we develop a novel dynamic GCN to adaptively extract structural information of HSI. Unlike the existing semisupervised GCNs for HSI classification, dynamic GCN can use supervised learning methods and generate discriminative vectors for the final classification. Specifically, our dynamic GCN consists of two graph convolutional layers, as shown in Fig. 2. The first layer performs regular graph convolution operations. For vertex feature $\mathbf{V} \in \mathbb{R}^{C\times D}$, the GCN layer aims to utilize an adjacency matrix $\mathbf{A}_f \in \mathbb{R}^{C\times C}$ and learnable parameters $\mathbf{W}_f$ to upstate the values of $\mathbf{V}$. Formally, this process can be formulated as

$$\mathbf{V}' = \sigma\left(\mathbf{A}_f \mathbf{V} \mathbf{W}_f\right) \tag{2}$$

where the adjacency matrix $\mathbf{A}$ records the relationship between each node. It is worth noting that $\mathbf{A}_f$ and $\mathbf{W}_f$ in the first layer are randomly initialized and learnable in training. $\sigma(\cdot)$ is an activation function, which is utilized to improve the nonlinearity of the network. In this article, we use the LeakyReLU activation function. It can be expressed as

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \eta x, & x < 0 \end{cases} \tag{3}$$

where η is the negative slope coefficient and is set to 0.01.

For the second layer, we introduce the adjacency matrix $\mathbf{A}_s$ to update the node $\mathbf{V}'$. $\mathbf{A}_s \in \mathbb{R}^{C\times C}$ is obtained from the encoded $\mathbf{V}$, which is different from the first layer that is fixed after training. $\mathbf{A}_s$ can be dynamically updated as the input features change. With the feature vectors $\mathbf{E}$, $\mathbf{F}$, and $\mathbf{G}$ transformed by the fully connected network, we have

$$\alpha_{ji} = \frac{\exp\left(\mathbf{E}_i \cdot \mathbf{F}_j\right)}{\sum_{i=1}^{K} \exp\left(\mathbf{E}_i \cdot \mathbf{F}_j\right)} \tag{4}$$

$$\mathbf{A}_s = \beta \sum_{i=1}^{K} \left(\alpha_{ji} \mathbf{G}_i\right) + \mathbf{V}_j \tag{5}$$

where β is a learnable parameter. Thus, different $\mathbf{A}_s$ values are generated for each patch, which greatly enhances the expressiveness of the model and reduces the risk of overfitting. Formally, the output $\mathbf{V}''$ can be defined as

$$\mathbf{V}'' = \sigma\left(\mathbf{A}_s \mathbf{V}' \mathbf{W}_s\right) \tag{6}$$

where $\mathbf{W}_s$ represents the learnable parameters and $\sigma(\cdot)$ denotes the LeakyReLU activation function. Overall, the FDGC can capture content-aware category dependencies with the help of the dynamic graph convolutional layer.

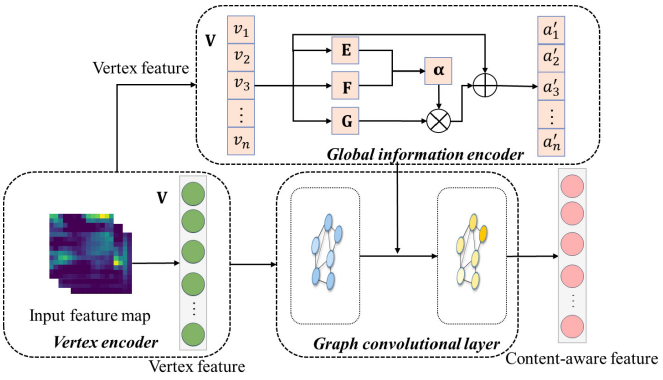


Fig. 2. Flowchart of the dynamic GCN module, where v_n and v_n' are of the (n)th element of the vertex feature before and after the global information encoding. $\mathbf{E}$, $\mathbf{F}$, and $\mathbf{G}$ are the feature vectors transformed by the fully connected network. α is the product of $\mathbf{E}$ and $\mathbf{F}$. The operator $\oplus$ is vector summation, and $\otimes$ is vector multiplication.

B. Convolutional Neural Network

It is a deep feedforward neural network with sparse interactions, parameter sharing, and equivariant representations, which can effectively improve the problem of fully connected networks with too many parameters and failure to maintain local invariant features [56]. CNN has been successfully applied in many computer vision tasks. In recent years, the CNN-based architecture has been successfully used in the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

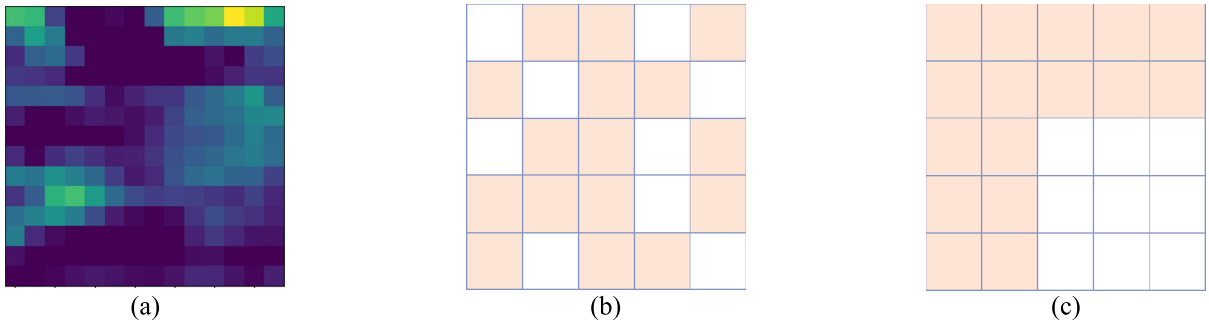


Fig. 3. Schematic of the DropBlock module randomly discarding semantic information. (a) Feature map is given. If light orange squares are used to indicate the feature values contained in the feature map and white squares are used to indicate the deactivated feature values, (b) represents the feature map obtained using random deactivation and (c) represents the feature map obtained using semantic deactivation.

HSI classification. Therefore, in this work, we utilize the spectral–spatial CNN to extract the features of HSI. The process can be formulated as

$$\mathbf{h}_j^{l+1} = \sigma\left(\sum_{i=1}^{M} \mathbf{h}_i^l * w_{ij}^{l+1} + b_j^{l+1}\right) \tag{7}$$

where $*$ refers to the convolution operator. The matrix $\mathbf{h}_i^l$ and $\mathbf{h}_j^{l+1}$ are the feature maps of the current layer and the next layer, respectively. w_{ij}^{l+1} and b_j^{l+1} denote the filters and the bias vector of the $(l+1)$th convolution layer, respectively. $\sigma(\cdot)$ denotes the LeakyReLU activation function.

C. Regularization

1) DropBlock Regularization: DropBlock is a simple regularization method (e.g., dropout). Given a feature map Fig. 3(a), Dropout randomly drops discontinuous neurons. As shown in Fig. 3(b), white color indicates dropped neurons, and light orange color indicates normal neurons. Nevertheless, since neighboring neurons usually have similar features, the network also learns the same information from the vicinity of the dropped activation units. Fig. 3(c) represents the random discard contiguous region used by DropBlock. By dropping out a portion of the whole adjacent area, the network will focus on learning other features to achieve correct classification and thus exhibit better generalization. DropBlock takes two main parameters: drop block size s and drop probability γ. In the present experiment, we set s to 3 and γ to 0.01 because the patch size is relatively small compared with the conventional RGB image size. Experimentally, the DropBlock module is shown to slightly increase the accuracy of the model.

2) Label Smoothing Regularization: Since HSI classification usually employs a cross-entropy loss function. However, this can cause two problems, including overfitting and incorrect learning labels. First, the neural network will remember all positive and negative samples, which is unpractical for the infinite number of samples in the real world. Second, manual labeling will inevitably produce errors. The calculation of the cross-entropy loss on the wrong labels will lead to divert optimization results.

Label smoothing regularization adds noise by soft one hot, which reduces the weight of the class of real sample labels in computing the loss function and ultimately has the effect of suppressing overfitting. Consider a prior distribution of label $u(c)$, independent training instance $x \in \mathbf{X}$, and a label smoothing coefficient ε. For a training instance with real labels $y \in \mathbf{Y}$, the probability distribution of the label distribution $q(c|x)$ after adding label smoothing becomes

$$q'(c|x) = (1-\varepsilon)\cdot q(c|x) + \varepsilon u(c) \tag{8}$$

$$u(c) = \frac{1}{C} \tag{9}$$

where N denotes the total number of categories and $q'(c|x)$ is a mixture of the original ground truth distribution y_i and the fixed distribution $u(c)$, with weights $1-\varepsilon$ and ε, respectively. On our Indian Pines and Salinas datasets experiments with C equaling to 16, we set $u(c)$ to $1/16$ and ε to 0.01. For Houston 2018 experiments with C equaling to 20, we set $u(c)$ to $1/20$ and same ε. We will discuss this parameter setting in detail in Section IV.

D. Feature Fusion and Classification

To fully explore the information contained in HSI, we use three branches to extract HSI features from different perspectives. As shown in Fig. 1, the outputs of the three branches are $\mathbf{O}_1$, $\mathbf{O}_2$, and $\mathbf{O}_3$. We flatten the feature map of the output of branch one to obtain $\mathbf{O}_1$ and perform the global average pooling operation in branch two to get $\mathbf{O}_2$. $\mathbf{O}_3$ is obtained from the dynamic graph convolution module. The features obtained from these three different perspectives are fused and then classified. The output feature $\mathbf{O}$ can be represented as

$$\mathbf{O} = \mathbf{O}_1 || \mathbf{O}_2 || \mathbf{O}_3 \tag{10}$$

where the operator $||$ denotes concatenating features along the spectral dimension. To optimize the proposed FDGC, we use the cross-entropy loss function as

$$L = -\frac{1}{K}\sum_{k=1}^{K}\sum_{c=1}^{C} \mathbf{y}_k^c \log\left(\tilde{\mathbf{y}}_k^c\right) \tag{11}$$

where K is the number of samples in a mini-batch, C is the total number of categories of the dataset, and $\mathbf{y}_k^c$ and $\tilde{\mathbf{y}}_k^c$ are the ground truth and predict labels of the (k)th of class c, respectively. The model weights and bias are updated by backpropagation and stochastic gradient descent.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

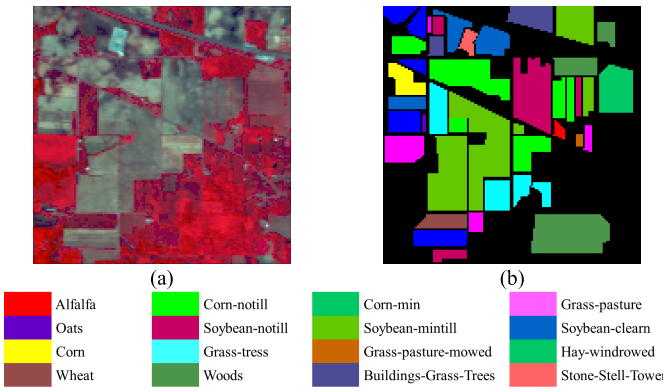


Fig. 4. Visualization of Indian pines dataset. (a) False-color map. (b) Ground truth map.

TABLE I

LAND COVER CATEGORIES AND THE DATASET DIVISION FOR EACH CATEGORY OF INDIAN PINES DATASET

No.	Class	Training	Test	Total
1	Alfalfa	15	31	46
2	Corn-notill	100	1328	1428
3	Corn-min	100	730	830
4	Corn	100	137	237
5	Grass/Pasture	100	383	483
6	Grass/Trees	100	630	730
7	Grass/pasture-mowed	15	13	28
8	Hay-windrowed	100	378	478
9	Oats	15	5	20
10	Soybeans-notill	100	872	972
11	Soybeans-min	100	2355	2455
12	Soybeans-clean	100	493	593
13	Wheat	100	105	205
14	Woods	100	1165	1265
15	Bldg-Grass-Tree-Drives	100	286	386
16	Stone-steel towers	15	78	93
	Total	1260	8989	10249

IV. EXPERIMENTS

In this section, we first give a brief description of the experimental settings. Then, comparative experiments are designed to evaluate the performance of the model. Finally, the various factors affecting the model performance are discussed.

A. Hyperspectral Datasets

1) Indian Pines Dataset: Indian Pines dataset [57] was collected by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor, representing 145 × 145 pixels, 20-m spatial resolution vegetation classification images, as shown in Fig. 4. The dataset has 200 spectral reflectance bands after the bands covering the regions of water absorption in the 0.4–2.5-μm region of the visible and infrared spectra are deleted. A total of 10 249 pixels from 16 classes are applied for our experiment, as shown in Table I.

2) Salinas Dataset: Salinas dataset [28] was acquired by the AVIRIS sensor. It was collected in the Salinas Valley in California. This dataset contains 512 scan lines, and each line has 217 pixels, as shown in Fig. 5. The spatial resolution is about 3.7 m. After deleting the low signal-to-noise ratio and water absorption bands, a total of 204 channels remained,

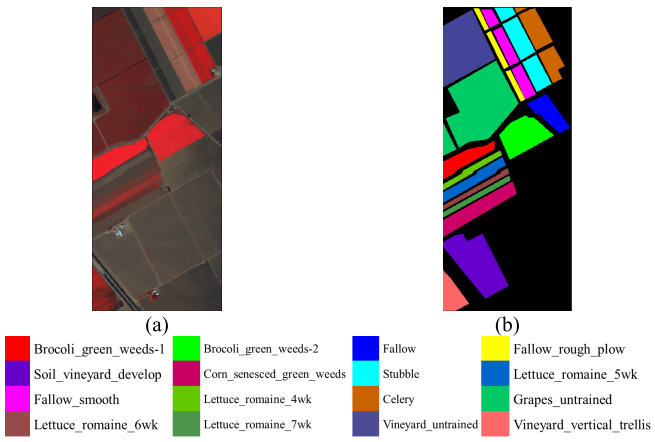


Fig. 5. Visualization of Salinas dataset. (a) False-color map. (b) Ground truth map.

TABLE II

LAND COVER CATEGORIES AND THE DATASET DIVISION FOR EACH CATEGORY OF SALINAS DATASET

No.	Class	Training	Test	Total
1	Broccoli-green-weeds-1	100	1909	2009
2	Broccoli-green-weeds-2	100	3626	3726
3	Fallow	100	1876	1976
4	Fallow-rough-plow	100	1294	1394
5	Fallow-smooth	100	2578	2678
6	Stubble	100	3859	3959
7	Celery	100	3479	3579
8	Grapes-untrained	100	11171	11271
9	Soil-senesced-develop	100	6103	6203
10	Corn-weeds	100	3178	3278
11	Lettuce-romaine-4wk	100	968	1068
12	Lettuce-romaine-5wk	100	1827	1927
13	Lettuce-romaine-6wk	100	816	916
14	Lettuce-romaine-7wk	100	970	1070
15	Vinyard-untrained	100	7168	7268
16	Vinyard-vertical-trellis	100	1707	1807
	Total	1600	52529	54129

which are in the 0.36–2.5-μm region of the visible and infrared spectra. A total of 54 129 pixels from 16 classes are used in this experiment, as shown in Table II.

3) Houston 2018 Dataset: Houston 2018 dataset [56] was gathered by the Compact Airborne Spectrographic Imager (CASI) 1500 over the area of the University of Houston on February 16, 2017. This dataset has 48 spectral bands in the spectral range of 0.38–1.05 μm. The ground sampling distance of the image is 1.3 m. In this article, we utilize the training portion of the whole dataset, which is distributed by the University of Houston and the Image Analysis and Data Fusion Technical Committee for the data fusion contest. The image comprises complex building, roads, and other areas, and has an image size of 601 × 2384 pixels, as shown in Fig. 6. Twenty ground truth classes with a total of 504 712 samples are considered in the experiment, as shown in Table III.

B. Experimental Settings

All the activation functions in FDGC are the LeakyReLU and use the Adam optimizer with label smoothing. The number of training iterations is 100 and the batch size is 128.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

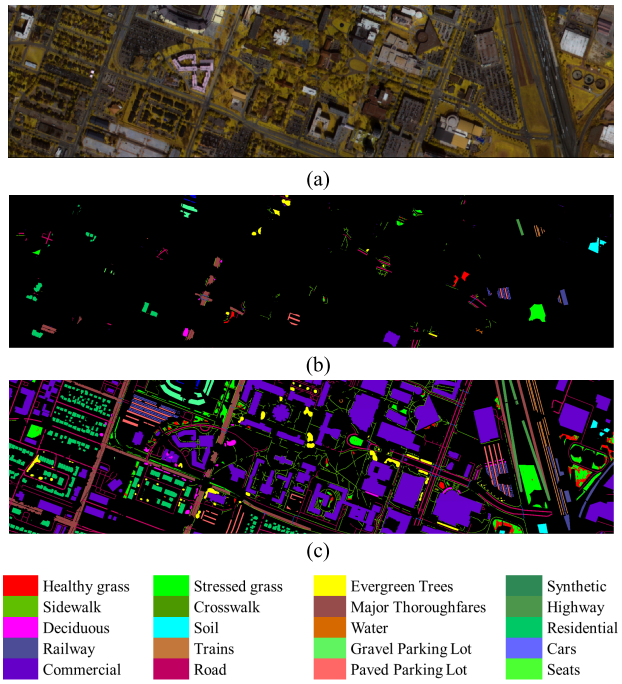


Fig. 6. Visualization of Houston 2018 dataset. (a) False-color map. (b) Training samples' ground truth map. (c) Test samples' ground truth map.

TABLE III
LAND COVER CATEGORIES AND THE DATASET DIVISION FOR EACH CATEGORY OF HOUSTON 2018 DATASET

No.	Class	Training	Test	Total
1	Healthy grass	1458	8341	9799
2	Stressed grass	4316	28186	32502
3	Artificial turf	331	353	684
4	Evergreen trees	2005	11583	13588
5	Deciduous trees	676	4372	5048
6	Bare earth	1757	2759	4516
7	Water	147	119	266
8	Residential buildings	3809	35953	39762
9	Non-residential buildings	2789	220895	223684
10	Roads	3188	42622	45810
11	Sidewalks	2699	31303	34002
12	Crosswalks	225	1291	1516
13	Major thoroughfares	5193	41165	46358
14	Highways	700	9149	9849
15	Railways	1224	5713	6937
16	Paved parking lots	1179	10296	11475
17	Unpaved parking lots	127	22	149
18	Cars	848	5730	6578
19	Trains	493	4872	5365
20	Stadium seats	1313	5511	6824
	Total	34477	470235	504712

The learning rate decreases as the number of iterations increases. The initial learning rate is 0.001, which is multiplied by 0.1 at the 50th and 83rd iterations. The spatial patch size is set to 19 × 19 and dimension is reduced to 32 by PCA. The specific network parameters of FDGC are shown in Table IV. Our experiments are implemented with Python-3.8.5 and PyTorch-1.7.1. The environment consists of an i7-10700K CPU with 32 GB and a NVIDIA GTX-2060 graphical processing unit with 6 GB.

In order to alleviate the influence of random factors, all the experiments are repeated five times. The mean values of the

TABLE IV
FDGC NETWORK CONFIGURATION

Layers	FDGC
Convolution Layer	3×3@32
Convolution Layer	3×3@64
Max pooling Layer	2×2
Branch one	[3×3@64; 3×3@64]
	Flatten
Branch two	[3×3@64; 3×3@128]
	Global average pooling
Branch three	[64; class number]
Fully connection Layer	1024
Fully connection Layer	256
Classification Layer	Softmax

overall accuracy (OA), the average accuracy (AA), the kappa coefficient, and standard variance are taken as the evaluation indices. Precisely, OA is the ratio of the sample size of correctly predicted categories to the total tested sample size, while AA is the average of the accuracy of all tested categories. The size of the standard deviation can assess the stability of the model. Furthermore, the kappa coefficient is a statistic widely used to measure the consistency of multiclassification tasks.

C. Comparison of Classification Performance

In this section, seven supervised classification methods are used to compare with the proposed FDGC, including the machine learning-based benchmark algorithm SVM and seven DL methods. For supervised machine learning methods, since the kernel trick can significantly improve the performance of SVM, we use the radial basis functions (RBF) kernel, which is one of the most used kernels in SVM. For supervised DL methods, SpectralFormer [58] applies transformers to HSI classification, which efficiently mine and represent the sequence attributes of spectral signatures. 3-D CNN [59] is a classical DL benchmark method. Deep CNN (DCNN) [60] extends the depth of 3-D CNN and enhances its ability to extract spatial information. Spectral–spatial residual network (SSRN) [61] introduces the residual structure to HSI classification, which alleviates the declining-accuracy phenomenon. Deep feature fusion network (DFFN) [62] is a 2-D CNN network that fuses the outputs of different hierarchical layers with the help of residual structure. Hybrid spectral CNN (HybridSN) [63] adds an attention mechanism to the CNN network. Spectral partitioning residual network (SPRN) [64] is a CNN-based spectral partitioning residual network. These DL models mine HSI feature information from different perspectives and represent the latest advances in the HSI classification. To make a fair comparison, the spatial patch size and dimension are set to the same for all DL-based methods, while SVM adopts serialized original data.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON RESULTS OBTAINED ON THE INDIAN PINS DATASET

Class	SVM	SpectralFormer	DCNN	3DCNN	SSRN	DFFN	HybridSN	SPRN	FDGC
1	76.13±6.64	90.32±5.4	93.69±4.56	93.02±8.68	80.13±17.19	90.56±6.15	**100±0**	75.59±12.94	85.4±9.87
2	69.37±1.93	83.72±3.15	95.75±0.42	88.46±2.94	96.81±1.44	97.66±2.2	95.37±1.75	**97.87±0.56**	97.85±1.1
3	72.38±4.34	89.01±2.65	98.03±0.71	92.74±2.97	99.04±0.8	98.45±0.83	97.37±2.42	99.17±1.25	**99.59±0.35**
4	87.59±4.33	97.66±1.93	95.76±2.46	92.24±6.8	96.89±3.61	96.06±3.58	98.19±2.3	96.63±3.75	**99.42±0.54**
5	92.01±2.59	96.55±1.72	97.31±2.65	95.41±1.24	98.12±1.26	98.04±0.57	96.91±1.7	97.43±1.94	**98.86±0.6**
6	94.67±0.51	98.67±0.34	**96.28±2.39**	96.67±1.31	93.31±4.4	93.79±3.19	93.92±3.14	93.75±3.89	94.71±2.57
7	92.31±4.87	95.38±6.15	93.55±11.54	98.81±2.66	98.57±2.86	98.57±2.86	98.57±2.86	**100±0**	**100±0**
8	97.51±1.15	99.68±0.31	99.69±0.28	98.09±1.87	98.42±1.82	98.19±1.17	99.22±0.69	**99.84±0.13**	99.38±1.23
9	96±8	100±0	93.75±13.98	**100±0**	79.09±25.65	79.55±28.82	75.88±21.39	66.59±28.03	92.5±15
10	79.79±4.46	88.51±1.63	94.13±2.18	92.01±3.75	95.45±3.28	94.51±2.73	94.97±1.69	95.9±1.48	**97.65±1.08**
11	65.8±1.59	81.39±3.81	98.88±0.59	97.46±1.41	**99.65±0.21**	99.23±0.45	98.47±0.6	99.17±0.46	99.58±0.19
12	82.39±1.54	93.14±0.89	93.57±2.58	85.48±3.51	**98.84±0.95**	98.59±1.23	95.58±2.48	97.21±1.87	97±2.96
13	98.29±1.26	**98.86±1.11**	98.49±2.23	96.89±3.36	92.63±5.63	92.07±10.04	95.06±8.18	91.41±7.06	96.79±3.24
14	90.35±1.35	96.12±0.86	98.93±0.53	98.72±1.16	**99.24±0.74**	97.82±1.16	98.93±0.76	99.19±0.92	99.11±0.4
15	72.87±1.66	96.36±2.05	93.11±3.26	92.22±3.71	95.29±1.4	96.62±2.52	93.38±2.81	95.54±4.31	**96.91±1.62**
16	95.38±1.03	91.28±6.71	90.2±9.02	95.46±3.66	96.85±3.37	95.26±4.71	**97.92±1.04**	96.74±2.95	94.11±6.18
OA (%)	78.07±0.85	89.3±1.36	96.96±0.52	94.08±0.98	97.59±0.74	97.36±0.55	96.83±0.79	97.61±0.56	**98.27±0.28**
AA (%)	85.18±0.88	93.54±1.43	95.69±1.33	94.61±0.73	94.9±1.86	95.31±1.99	95.61±1.95	93.88±2.7	**96.81±1.02**
kappa×100	75±0.98	87.73±1.55	96.49±0.6	93.18±1.12	97.22±0.85	96.95±0.64	96.33±0.91	97.24±0.64	**98±0.33**

Note that here, we do not use semisupervised GCN as the comparison algorithm for two reasons.

1) Semisupervised learning makes full use of unlabeled data, which is different from supervised learning that only uses labeled data for training models. This feature allows for a large difference in model training time and training performance [57].

2) Large amount of unlabeled data consumes a lot of video memory. Even with the help of the superpixel segmentation algorithm often employed by such methods to construct graph data, it is still difficult to scale to the Salinas and Houston 2018 datasets in the same experimental setting.

For the Indian Pine and Salinas datasets, the training set is a random selection of 100 sample points, and the remaining sample points are used as the test set. Tables I and II provide information about the corresponding training and test samples. For the Houston 2018 dataset, training and testing samples are officially predefined according to Table III. Therefore, the results obtained with this benchmark dataset may be considered more reliable and stable.

We quantitatively evaluate the classification performance on the Indian Pines dataset in Table V, where the proposed FDGC outperforms all the comparison approaches in most cases. The best results for each metric are shown in bold. Specifically, SVM gets much less accuracy than other approaches, showing the importance of spatial information. SpectralFormer also obtains a relatively poor accuracy. This might be that transformer needs large data training. The accuracy of the 3-D CNN is about 2%–4% gap compared with other spatial–spectral combined methods, which can be attributed to the positive impact of the architecture and specific module design. Both DCNN and HybridSN obtain high AA, which may be due to their subtle information extraction ability and the addition of attention mechanism. Furthermore, since the intrinsic structure and spatial–spectral information of HSI are fully utilized, the classification accuracies obtained by FDGC are higher than methods, and the three branches make it have the best robustness.

More visually, Fig. 7 shows the visualized classification results of all these approaches in a single experiment on the Indian Pines dataset. It can be seen from Fig. 7 that the classification maps generated by DL-based methods are smoother than SVM. The classification map of SVM has serious salt and pepper noise, while the result of FDGC is both smoother and more consistent with the ground truth map than all the comparison methods. All the observations validate the superiority of our methods and the rationality of integrating GCN and CNN modules together.

In the following, the detailed classification results on the Salinas dataset are shown in Table VI. The Salinas dataset is a sample-balanced dataset compared to the Indian Pines dataset, so all algorithms achieved better accuracy than on the Indian Pines dataset. However, perhaps because HSI has intraclass variability and interclass similarity phenomena, the SVM based on spectral feature extraction still cannot effectively distinguish these features. On class 1 Broccoli-green-weeds-1 and class 9 Soil-vineyard-develop, some DL-based methods obtained relatively low-class accuracies, which may be because the more severe mixed pixels in both classes result in more intraclass spectral differences and less spatial differences. 3-D CNN and HybridSN achieve good accuracies on classes 9 and 1, respectively, but this causes severe overfitting,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

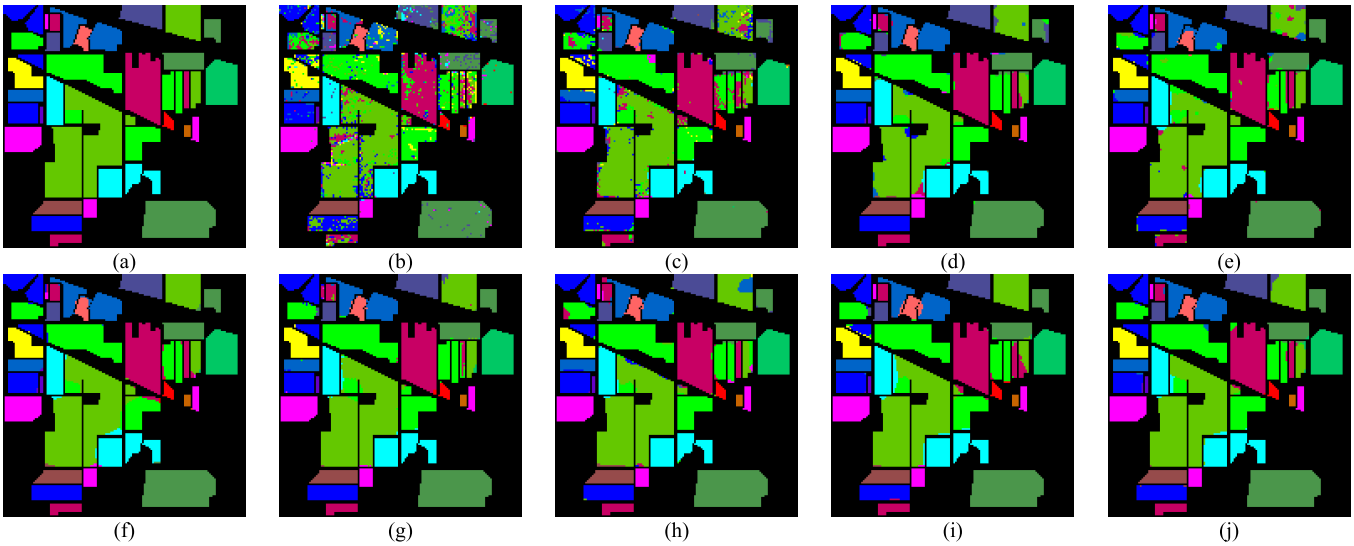


Fig. 7. Classification maps of different methods on the Indian Pines dataset. (a) Ground truth. (b) RBF-SVM. (c) SpectralFormer. (d) DCNN. (e) 3-D CNN. (f) SSRN. (g) DFFN. (h) HybridSN. (i) SPRN. (j) FDGC.

TABLE VI
COMPARISON RESULTS ON THE SALINAS DATASET

Class	SVM	SpectralFormer	DCNN	3DCNN	SSRN	DFFN	HybridSN	SPRN	FDGC
1	98.24±0.8	99.97±0.04	99.76±0.51	99.97±0.06	**100±0**	99.99±0.02	99.95±0.12	99.99±0.02	**100±0**
2	99.56±0.22	99.96±0.05	99.78±0.5	99.96±0.09	**100±0**	**100±0**	99.97±0.06	**100±0**	**100±0**
3	99.54±0.14	99.98±0.03	99.93±0.08	99.83±0.21	**100±0**	99.99±0.02	99.99±0.02	**100±0**	99.98±0.04
4	**99.69±0.08**	99.55±0.2	99.01±0.74	99.37±0.28	97.59±1.77	98.35±1.4	99.68±0.37	96.25±2.93	98.32±0.91
5	96.83±0.47	98.52±0.62	99.84±0.24	99.76±0.28	**99.91±0.14**	99.86±0.16	99.77±0.16	99.88±0.08	99.88±0.16
6	99.58±0.24	99.99±0.01	99.88±0.15	99.98±0.04	99.97±0.04	99.99±0.03	**100±0**	99.93±0.08	**99.99±0.01**
7	99.48±0.24	99.75±0.17	99.94±0.09	99.98±0.05	**100±0**	**100±0**	99.99±0.03	**100±0**	**100±0**
8	76.18±7.47	84.94±2.94	99.72±0.21	99.05±0.52	99.65±0.58	**99.94±0.09**	99.61±0.28	**99.94±0.09**	99.77±0.15
9	98.29±1.4	99.98±0.02	99.99±0.03	99.88±0.19	**100±0**	100±0.01	99.97±0.04	99.99±0.02	99.97±0.06
10	92.31±1.05	98.31±0.83	99.18±0.67	97.94±3.37	100±0.01	98.82±2.27	99.75±0.33	**100±0**	99.99±0.02
11	97.17±1.49	99.94±0.08	98.58±1.35	99.41±0.68	**100±0**	99.95±0.08	99.46±0.98	**100±0**	99.94±0.12
12	99.86±0.11	99.95±0.08	**100±0**	99.78±0.36	**100±0**	**100±0**	99.99±0.02	**100±0**	99.87±0.24
13	98.09±0.63	99.68±0.3	99.36±1.01	99.56±0.55	99.84±0.15	99.69±0.2	99.88±0.1	99.82±0.31	**99.93±0.1**
14	96.39±1.12	99.48±0.33	98.98±1.92	99.8±0.33	99.86±0.12	99.52±0.56	**99.88±0.22**	99.64±0.47	99.86±0.29
15	72.17±3.22	91.48±2.6	95.8±1.03	92.9±2.95	98.8±1.2	97.98±1.53	95.87±1.49	98.7±0.5	**98.99±0.51**
16	98.64±0.19	99.21±0.43	98.57±2	92.9±2.95	**100±0**	**100±0**	**100±0**	**100±0**	**100±0**
OA (%)	89.94±1.24	95.38±0.41	99.12±0.22	98.54±0.68	99.69±0.26	99.56±0.25	99.27±0.22	99.69±0.1	**99.75±0.09**
AA (%)	95.13±0.29	98.17±0.11	99.27±0.22	99.2±0.36	99.73±0.13	99.63±0.2	99.61±0.11	99.64±0.18	**99.78±0.09**
kappa×100	91.49±1.06	94.85±0.45	99.01±0.25	98.38±0.75	99.65±0.29	99.51±0.27	99.19±0.24	99.65±0.11	**99.72±0.1**

resulting in low overall classification accuracies. Compared to the others CNN-based algorithms, the proposed FDGC achieves the highest accuracy among all the methods. FPGC fuses the features from dynamic GCN and CNN to achieve the fusion structure information and spatial details, which also allows us to achieve the smallest standard deviation. The predicted maps obtained by various methods in a single experiment are visually shown in Fig. 8. As expected, FDGC generates a more realistic classification map, which is in more accordance with the ground truth map.

Table VII and Fig. 9 report the quantitative and visualized results of all methods on the Houston 2018 datasets. It is worth stating that SVM produces the same classification results for the same input, while the DL-based models' results fluctuate due to random initialization of the weights. Therefore, for the Houston 2018 dataset, the standard deviation of the SVM is equal to 0. SVM, 3-D CNN, DCNN, and HybridSN have similar performance in this dataset with an accuracy of about 60%. This may be because the model has strong constraints and cannot make full use of shallow and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

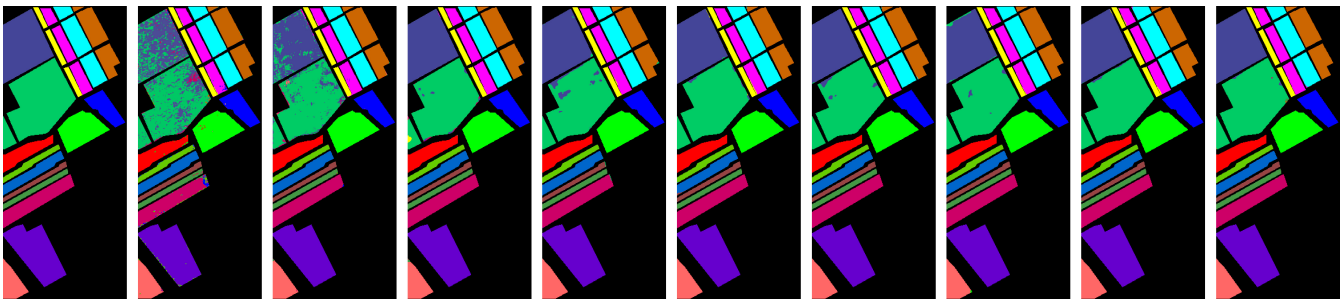


Fig. 8. Classification maps of different methods on the Salinas dataset. (a) Ground truth. (b) RBF-SVM. (c) SpectralFormer. (d) DCNN. (e) 3-D CNN. (f) SSRN. (g) DFFN. (h) HybridSN. (i) SPRN. (j) FDGC.

TABLE VII
COMPARISON RESULTS ON THE HOUSTON 2018 DATASET

Class	SVM	SpectralFormer	DCNN	3DCNN	SSRN	DFFN	HybridSN	SPRN	FDGC
1	**90.54**	69.84±3.55	41.91±4.63	44.29±4.95	44.34±7.95	41.1±7.37	46.45±8.24	48.18±6.61	56.66±3.49
2	**82.51**	81.87±2.2	55.53±6.02	67.24±4.78	67.27±5.33	70.47±6.07	63.62±5.41	57.6±3.78	77.89±3.55
3	84.86	**99.49±0.21**	13.03±11.27	40.25±16.78	53.53±34.92	44.85±32.16	6.07±1.34	66.09±35.33	86.14±9.22
4	75.43	**96.22±1.09**	74.08±6.43	82.95±2.43	78.39±3.49	81.72±8.19	75.66±7.81	74.85±3.75	82.98±2.18
5	17.22	56.04±3.87	54.04±3.93	61.67±8.88	67.52±23.16	53.64±15.65	68.15±7.6	**81.21±5.31**	68.28±3.89
6	15.15	30.06±0.43	43.97±5.68	74.52±11.1	57.09±18.4	53.61±7.51	76.55±13.54	**86.33±6.33**	77.52±11.09
7	33.03	**75.8±15.01**	23.83±19.15	35.88±33.01	5.03±8.05	13.02±25.54	1.96±0.7	0.09±0.14	9.89±11.32
8	40.27	**78.22±3**	48.18±5.8	40.62±3	66.11±1.57	67.69±1.87	55.89±4.4	71.07±3.87	73.11±5.69
9	92.93	56.94±4.28	92.4±0.71	92.13±1.12	92.85±0.39	93.1±0.7	92.58±1.14	93.21±0.72	**94.23±0.91**
10	43.08	**57.57±2.15**	43.59±2.75	45.49±3.07	48.78±1.22	51.75±4.71	50.06±1.97	48.18±1.27	44.07±3.21
11	28.56	61.69±3.58	24.04±2.49	26.72±0.43	33.62±2.17	**37.1±4.04**	25.88±2.59	31.18±1.79	29.4±3.12
12	0.00	6.03±1.36	**7.67±4.06**	19±9.42	2.99±0.81	1.48±0.36	2.83±1.07	6.61±5.1	6.16±3.93
13	45.96	52.66±3.67	57.86±4.08	59.96±2.81	68.42±1.21	65.29±2.49	62.59±2.54	69.17±3.45	**70.53±3.53**
14	59.94	40.99±3.62	49.06±8.11	53.57±7	56.63±9.02	53.05±9.49	49.97±8.31	**74.2±4.59**	60.16±12.22
15	83.95	61.53±2.39	71.02±6.56	61.33±12.14	67.54±15.33	75.56±9.15	**87.63±5.42**	58.79±12.23	84.11±6.03
16	57.20	**77.76±4.8**	64.94±5.35	65.78±7.36	73.55±3	70.53±3.67	68.65±4.35	47.48±6.72	73.99±4.17
17	1.73	**96.36±3.4**	1.74±0.91	5.53±5.21	15.04±25.83	10.48±14.63	1.36±0.62	22.06±24.84	14.06±10.42
18	23.18	**70.9±3.6**	25.33±1.54	29.2±8.71	39.34±14.97	35.33±9.8	34.45±8.43	27.88±7.45	41.52±6.39
19	33.60	**90.93±3**	46.89±7.28	73.88±6.06	80.32±6.96	79.45±6.4	75.08±13.45	62.71±17.7	69.72±15.85
20	25.19	**85.87±6.66**	36.33±10.14	37.04±15.64	79.08±11.94	48.42±8.14	44.71±18.16	63.94±19.12	49.06±14.9
OA (%)	57.89	62.05±1.68	56.33±1.01	57.34±1.77	69.79±1.3	69.33±0.52	61.42±2.83	68.29±0.62	**70.06±0.27**
AA (%)	46.72	**67.34±0.95**	43.77±1.07	50.85±1.66	54.87±3.54	52.38±1.96	49.51±1.49	54.54±1.56	58.47±1.25
kappa×100	49.36	54.22±1.56	47.3±0.97	48.79±1.63	60.99±1.41	60.84±0.58	52.75±2.78	59.26±0.65	**61.32±0.25**

deep features, leading to poor generalization ability. Although SpectralFormer has only an OA of 62.05%, it achieves an AA of 67.34%. It shows that a larger volume of data is what may bring out the power of the transformer. FDGC achieves better results and 1%–2% improvement over the SSRN, DFFN, and SPRN of the CNN-based methods. Moreover, FDCN achieves an OA of 86.14% on the artificial turf category, far exceeding other DL algorithms, which benefits from the following.

1) *The DropBlock Module:* DropBlock can automatically drop continuous semantic information, which forces the model to learn more general features.

2) *The Dynamic Module:* By adding dynamic GCN, the model can obtain more global spatial context and spatial details. These two points boost the classification performance.

On the Houston 2018 dataset, the accuracy obtained is clearly lower than obtained on the other two datasets using the same method. In addition to the difficulty of this dataset itself, another reason is the disjoint training and test set data partitioning method ensures that no pixels from the test set are smuggled into the training set. This data division significantly reduces the HSI pixels exposed to the model and places higher demands on the model generalization capability. As a result, the performance of all the methods drops observably.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

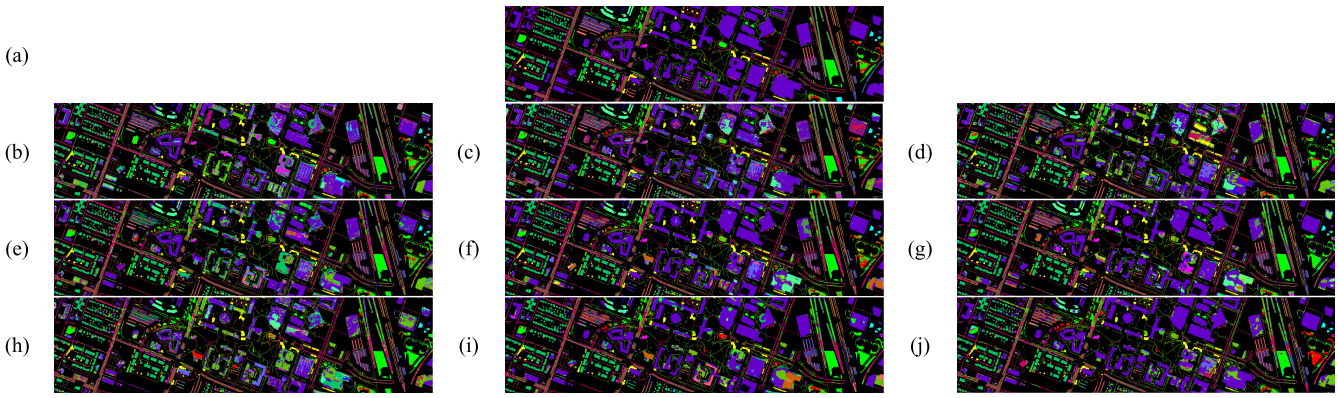


Fig. 9. Classification maps of different methods on the Houston 2018 dataset. (a) Ground truth. (b) RBF-SVM. (c) SpectralFormer. (d) DCNN. (e) 3-D CNN. (f) SSRN. (g) DFFN. (h) HybridSN. (i) SPRN. (j) FDGC.

TABLE VIII
RUNNING TIME OF DIFFERENT METHODS ON EACH DATASET IN SECONDS (S)

		SVM	Spectral Former	DCNN	3D-CNN	SSRN	DFFN	HybridSN	SPRN	FDGC
Network depth and type		/	/	4, Conv3d	3, Conv3d	11, Conv3d	25, Conv2d	3, Conv3d 1, Conv2d	10, group Conv2d	6, Conv2d
Indian Pines dataset	Train time(s)	**0.11**	77.82	38.16	51.16	148.45	57.92	75.43	93.54	**20.57**
	Test time(s)	1.90	2.36	**0.97**	1.43	1.95	1.31	1.60	1.93	**0.55**
Salinas dataset	Train time(s)	**0.06**	82.59	53.01	65.24	190.57	72.81	96.11	109.54	**24.99**
	Test time(s)	7.34	11.62	**6.34**	8.46	11.66	7.64	9.37	11.24	**3.05**
Houston 2018 dataset	Train time(s)	**14.67**	526.11	1055.30	1432.29	4120.12	1541.18	2022.92	2328.33	**471.60**
	Test time(s)	446.34	**32.93**	59.85	82.81	108.35	67.59	82.54	99.80	**34.02**

D. Analysis of the Running Time

In this section, we will discuss the time efficiency of models, which is an important indicator to evaluate algorithm practicability, as shown in Table VIII. The best values are highlighted in bold, and the suboptimal values are underlined. There are several factors that affect the time efficiency of neural networks. In the experimental scheme used in this article, we control for the same parameters settings, such as input, epoch, batch size, learning rate, and GPU. Therefore, the training and test time of the algorithm is mainly affected by the memory access cost [65]. As shown in Table VIII, SVM and SpectralFormer have unique characteristics. SVM is fast in training time but slow in test time. This is probably since GPU acceleration is not used. Some of the deep models outperform SVM in test speed due to GPU acceleration. SpectralFormer is slow in Indian Pines and Salinas datasets but fast in Houston 2018 dataset. It may be that only big data can harness the power of transformer parallel computing. In terms of CNN-based DL method, we find that the network depth and types are the main factors affecting the speed. The time taken by DFFN with 25 conv2d and 3-D CNN with 3 Conv3d is comparable, which indicates that the Conv3d operation is very time-consuming.

The FDGC uses GCN to extract the topological structure information of HSIs. This breaks the limitation of CNN local field of perception, and all information of each patch is accessible at each layer. Therefore, only a small memory access cost is required to fully utilize the information contained in each patch. The FDGC uses a three-branch parallel training with a shallower network structure, so its training and test speed is substantially ahead of the compared deep models. For example, compared with the recently proposed SPRN, we only need about 1/5 of its training time to achieve similar accuracy, which shows the promise of the FDGC for applications in the industrial world.

TABLE IX
ABLATION STUDY ON THE INDIAN PINES DATASET. BRANCH ONE, BRANCH TWO, AND BRANCH THREE AS SHOWN IN FIG. 1

No.	Branch one	Branch two	Branch three	OA
1	√			97.31±1.08
2		√		97.64±0.35
3			√	97.50±0.50
4	√	√		97.05±0.56
5	√		√	97.51±0.65
6		√	√	97.88±0.24
7	√	√	√	98.27±0.28

E. Analysis of the Different Modules in FDGC

We designed a series of ablation experiments to study the contribution of different branches to the model on the Indian Pines dataset and Salinas dataset. The accuracy of ablation experiment is shown in Tables IX–XI. It can be observed that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

TABLE X

ABLATION STUDY ON THE SALINAS DATASET. BRANCH ONE, BRANCH TWO, AND BRANCH THREE AS SHOWN IN FIG. 1

No.	Branch one	Branch two	Branch three	OA
1	√			98.74±0.35
2		√		99.23±0.17
3			√	99.46±0.33
4	√	√		99.21±0.28
5	√		√	99.71±0.10
6		√	√	99.64±0.17
7	√	√	√	99.75±0.09

TABLE XI

ABLATION STUDY ON THE HOUSTON 2018 DATASET. BRANCH ONE, BRANCH TWO, AND BRANCH THREE AS SHOWN IN FIG. 1

No.	Branch one	Branch two	Branch three	OA
1	√			72.48±0.61
2		√		72.24±1.73
3			√	69.74±1.65
4	√	√		72.36±1.47
5	√		√	69.31±0.59
6		√	√	69.39±0.31
7	√	√	√	70.06±0.27

the accuracy of the model increases, and the variance of the model decreases as the number of fused features increases. On the Indian Pines dataset, the accuracy of the model increases by about 0.39%–0.96%. On the Salinas dataset, the FDGC has increased by up to 1.01%. Surprisingly, on the Houston 2018 dataset, the fusion model accuracy is lower than that of individual branches. This may be due to the disjoint data partitioning method adopted by the Houston 2018 data and the lack of diversity in the selection of sample points, resulting in the inability of GCN to model the structure information well. However, the model after the fusion of multiple features can significantly reduce the variance of the results and obtain more stable results. In conclusion, by fusing the features of the three branches, the proposed FDGC module exhibits superior performance compared with the branch network on the Indian Pines and Salinas datasets. On the Houston dataset, FDGC also achieves a competitive accuracy with the lowest variance.

F. Analysis of the Drop Block Ratio and Label Smoothing

The classification performance is based on the proposed model structure, hyperparameters, regularization, and so on. The impact of these factors on the model performance has been studied in many works [46]. In this section, we will discuss the effect of DropBlock and label smoothing regularization on the performance of the model on the Indian Pines dataset and the Salinas dataset. They are mainly controlled by three parameters: drop block size s, drop probability γ, and label smoothing coefficient ε.

The drop block size and drop probability together affect how much semantic information is dropped in the feature map. As s and γ increase, the amount of discarded semantic information increases; conversely, more feature information is retained. Randomly deactivated convolutional units can be

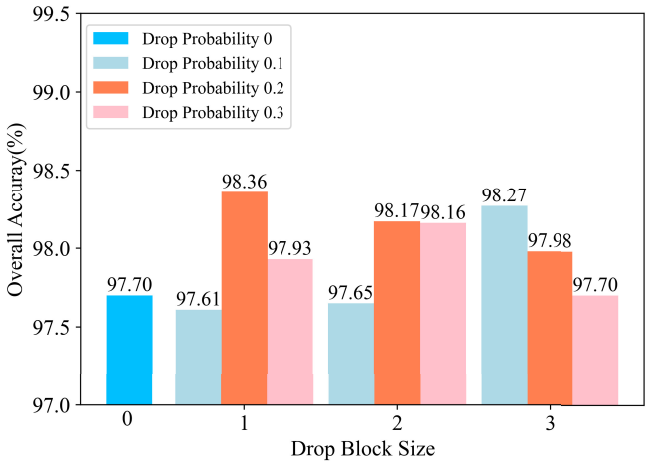


Fig. 10. Classification performance with different drop block sizes and drop probability on the India Pines dataset.

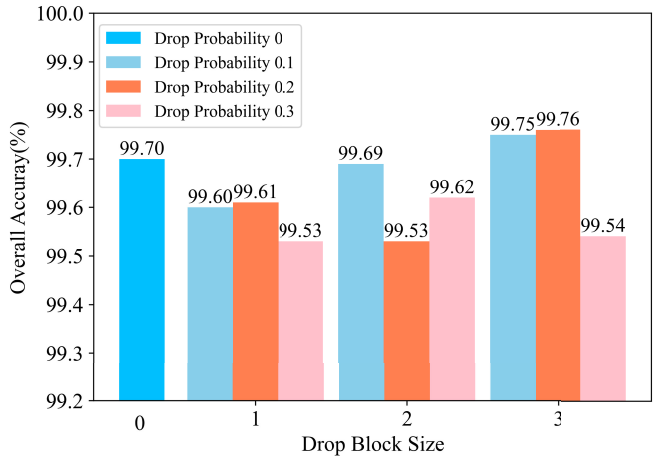


Fig. 11. Classification performance with different drop block sizes and drop probability on the Salinas dataset.

seen as a model integration, which will improve the generalization ability of the model. However, this may also lead to a decrease in feature diversity and weaken the performance of the model. Thus, s is empirically set to 0, 1, 2, and 3. γ is set to 0, 0.1, 0.2, and 0.3. Each experiment is repeated five times, and the mean OA indices are shown in Figs. 10 and 11. If we analyze the parameters s or γ individually, the model performance does not show a consistent trend. However, if we analyze them together, we can find that DropBlock does help in improving the performance of the model. Considering the performance on the two datasets, we experimentally set s and γ to 3 and 0.1, respectively.

To analyze label smoothing influence on our method, we set label smoothing coefficient ε to 0, 0.01, 0.04, 0.07, and 0.1 and test their classification accuracies on the Indian Pines dataset and Salinas dataset. As shown in Fig. 12, the mean and standard deviation of OA of the FDGC on both Indian Pines and Salinas datasets fluctuate with the increase of label smoothing coefficient, which indicates the effect of mixed pixels on the complexity of land cover labels. The larger standard deviation on the Indian Pines dataset may be due

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

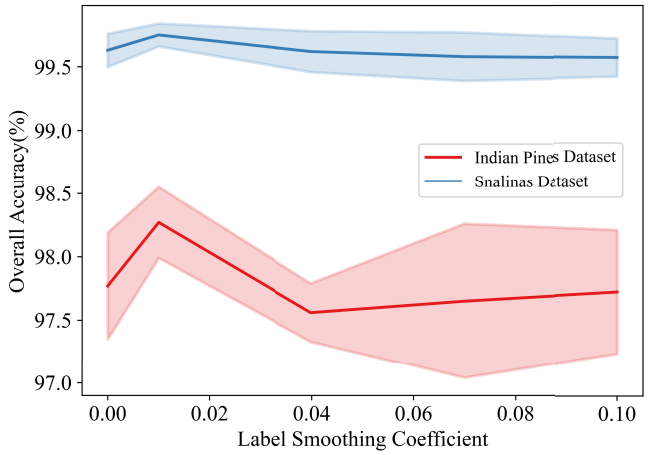


Fig. 12. Classification performance with different label smoothing coefficients on the Indian Pines and Salinas datasets.

to the unbalanced classes of the training samples in this dataset. Although larger label smoothing coefficients may lead to more noise being added to the labels, the proposed model still maintains relatively stable performance, which indicates the strong generalization capability of FDGC. The FDGC achieves the best performance on both datasets when $\varepsilon = 0.01$. Therefore, in our model, we set it to 0.01.

V. Conclusion

In this work, we propose an FDGC parallel network for HSI classification. FDGC first extracts generic features based on convolutional networks. Then, three branches are designed to parallelly extract three features. Finally, these three features are fused for classification. We design the vector encoder module to transform the feature map into vertex features and introduce the attention mechanism into the graph convolution networks to dynamically construct the adjacency matrix for each patch. These two modules together form the dynamic map convolution branch. To alleviate the overfitting problem caused by the small sample problem in the HSI classification, we apply a DropBlock layer to learn more generalizable features and label smoothing to reduce the interference of mixed pixels on the classification performance. Extensive experiments show that our proposed FDGC can obtain competitive results on three cases at a faster speed. It should state that the supervised learning approach is applied in our network, which allows the network to easily scale to large HSIs like Houston 2018 with better performance.

References

[1] L. Wang, J. Yan, L. Mu, and L. Huang, "Knowledge discovery from remote sensing images: A review," *WIREs Data Mining Knowl. Discovery*, vol. 10, no. 5, p. e1371, Sep. 2020.

[2] S. Mahesh, D. S. Jayas, J. Paliwal, and N. D. G. White, "Hyperspectral imaging to classify and monitor quality of agricultural materials," *J. Stored Products Res.*, vol. 61, pp. 17–26, Mar. 2015.

[3] S. Sabbah, "Remote sensing of gases by hyperspectral imaging: System performance and measurements," *Opt. Eng.*, vol. 51, no. 11, Jul. 2012, Art. no. 111717.

[4] J. Wang, A. Ma, Y. Zhong, Z. Zheng, and L. Zhang, "Cross-sensor domain adaptation for high spatial resolution urban land-cover mapping: From airborne to spaceborne imagery," *Remote Sens. Environ.*, vol. 277, Aug. 2022, Art. no. 113058.

[5] M. Shimoni, R. Haelterman, and C. Perneel, "Hypersectral imaging for military and security applications: Combining myriad processing and sensing techniques," *IEEE Geosci. Remote Sens. Mag.*, vol. 7, no. 2, pp. 101–117, Jun. 2019.

[6] L. O. Jimenez and D. A. Landgrebe, "Supervised classification in high-dimensional space: Geometrical, statistical, and asymptotical properties of multivariate data," *IEEE Trans. Syst., Man, Cybern., C (Appl. Rev.)*, vol. 28, no. 1, pp. 39–54, Feb. 1998.

[7] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[8] W. Xie, T. Jiang, Y. Li, X. Jia, and J. Lei, "Structure tensor and guided filtering-based algorithm for hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4218–4230, Jul. 2019.

[9] F. Luo, L. Zhang, B. Du, and L. Zhang, "Dimensionality reduction with enhanced hybrid-graph discriminant learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5336–5353, Aug. 2020.

[10] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "An augmented linear mixing model to address spectral variability for hyperspectral unmixing," *IEEE Trans. Image Process.*, vol. 28, no. 4, pp. 1923–1938, Apr. 2019.

[11] R. Feng, H. Li, L. Wang, Y. Zhong, L. Zhang, and T. Zeng, "Local spatial constraint and total variation for hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022.

[12] C. Rodarmel and J. Shan, "Principal component analysis for hyperspectral image classification," *Surv. Land Inf. Syst.*, vol. 62, no. 2, pp. 115–122, Jun. 2002.

[13] J. Xia, P. Ghamisi, N. Yokoya, and A. Iwasaki, "Random forest ensembles and extended multiextinction profiles for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 202–216, Jan. 2018.

[14] J. Peng, Y. Zhou, and C. L. P. Chen, "Region-kernel-based support vector machines for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 4810–4824, Sep. 2015.

[15] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

[16] Y. Tarabalka, M. Fauvel, J. Chanussot, and J. A. Benediktsson, "SVM- and MRF-based method for accurate classification of hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 7, no. 4, pp. 736–740, Oct. 2010.

[17] O. Rajadell, P. Garcia-Sevilla, and F. Pla, "Spectral–spatial pixel characterization using Gabor filters for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 4, pp. 860–864, Jul. 2013.

[18] L. Shen and S. Jia, "Three-dimensional Gabor wavelets for pixel-based hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 5039–5046, Dec. 2011.

[19] S. Jia *et al.*, "3-D Gaussian–Gabor feature extraction and selection for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 11, pp. 8813–8826, Nov. 2019.

[20] H. Huang, G. Shi, H. He, Y. Duan, and F. Luo, "Dimensionality reduction of hyperspectral imagery based on spatial–spectral manifold learning," *IEEE Trans. Cybern.*, vol. 50, no. 6, pp. 2604–2616, Jun. 2020.

[21] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[22] D. Hong *et al.*, "More diverse means better: Multimodal deep learning meets remote-sensing imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4340–4354, Apr. 2021.

[23] W. Han, R. Feng, L. Wang, and Y. Cheng, "A semi-supervised generative framework with deep learning features for high-resolution remote sensing image scene classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 23–43, Nov. 2018.

[24] A. Ma, Y. Wan, Y. Zhong, J. Wang, and L. Zhang, "SceneNet: Remote sensing scene classification deep learning network using multi-objective neural evolution architecture search," *ISPRS J. Photogramm. Remote Sens.*, vol. 172, pp. 171–188, Feb. 2021.

[25] H. Lee and H. Kwon, "Contextual deep CNN based hyperspectral classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Beijing, China, Jul. 2016, pp. 3322–3325.

[26] Y. Xu, L. Zhang, B. Du, and F. Zhang, "Spectral–spatial unified networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 5893–5909, Oct. 2018.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.

[27] Q. Gao, S. Lim, and X. Jia, "Hyperspectral image classification using convolutional neural networks and multiple feature learning," *Remote Sens.*, vol. 10, no. 2, p. 299, 2018.

[28] L. Qu, X. Zhu, J. Zheng, and L. Zou, "Triple-attention-based parallel network for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 2, p. 324, Jan. 2021.

[29] Z. Zhang, D. Liu, D. Gao, and G. Shi, "S^3Net: Spectral–spatial–semantic network for hyperspectral image classification with the multiway attention mechanism," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5505317.

[30] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203*.

[31] K. Xu, H. Huang, P. Deng, and Y. Li, "Deep feature aggregation framework driven by graph convolutional network for scene classification in remote sensing," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Apr. 15, 2021, doi: 10.1109/TNNLS.2021.3071369.

[32] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[33] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[34] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel-and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2020.

[35] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[36] J. Zhou *et al.*, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[37] E. Chien, J. Peng, P. Li, and O. Milenkovic, "Adaptive universal generalized PageRank graph neural network," 2020, *arXiv:2006.07988*.

[38] Z. Chen *et al.*, "Bridging the gap between spatial and spectral domains: A survey on graph neural networks," 2020, *arXiv:2002.11867*.

[39] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 29, 2016, p. 9.

[40] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[41] C. Zhuang and Q. Ma, "Dual graph convolutional networks for graph-based semi-supervised classification," in *Proc. World Wide Web Conf. World Wide Web (WWW)*, Lyon, France, 2018, pp. 499–508.

[42] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2020, pp. 639–648.

[43] D. Duvenaud *et al.*, "Convolutional networks on graphs for learning molecular fingerprints," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 28, 2015.

[44] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 30, 2017.

[45] J. Kukačka, V. Golkov, and D. Cremers, "Regularization for deep learning: A taxonomy," 2017, *arXiv:1710.10686*.

[46] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of tricks for image classification with convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Long Beach, CA, USA, Jun. 2019, pp. 558–567.

[47] M. Faramarzi, M. Amini, A. Badrinaaraayanan, V. Verma, and S. Chandar, "PatchUp: A regularization technique for convolutional neural networks," 2020, *arXiv:2006.07794*.

[48] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.

[49] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "DropBlock: A regularization method for convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 31, 2018.

[50] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, "Efficient object localization using convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 648–656.

[51] G. Larsson, M. Maire, and G. Shakhnarovich, "FractalNet: Ultra-deep neural networks without residuals," 2016, *arXiv:1605.07648*.

[52] S. Reed, H. Lee, D. Anguelov, C. Szegedy, D. Erhan, and A. Rabinovich, "Training deep neural networks on noisy labels with bootstrapping," 2014, *arXiv:1412.6596*.

[53] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Las Vegas, NV, USA, Jun. 2016, pp. 2818–2826.

[54] L. Xie, J. Wang, Z. Wei, M. Wang, and Q. Tian, "DisturbLabel: Regularizing CNN on the loss layer," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 4753–4762.

[55] C. Li, C. Liu, L. Duan, P. Gao, and K. Zheng, "Reconstruction regularized deep metric learning for multi-label image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 7, pp. 2294–2303, Jul. 2020.

[56] S. Jia *et al.*, "3-D Gabor convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022.

[57] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.

[58] D. Hong *et al.*, "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[59] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[60] A. Ben Hamida, A. Benoit, P. Lambert, and C. Ben Amar, "Three dimensional deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[61] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[62] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.

[63] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D-2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Jun. 2020.

[64] X. Zhang, S. Shang, X. Tang, J. Feng, and L. Jiao, "Spectral partitioning residual network with spatial attention mechanism for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[65] Y. Lee, J. Hwang, S. Lee, Y. Bae, and J. Park, "An energy and GPU-computation efficient backbone network for real-time object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops*, Long Beach, CA, USA, Jun. 2019, pp. 752–760.



Quanwei Liu received the B.S. degree in exploration technology and engineering from the Henan University of Engineering, Zhengzhou, China, in 2020. He is currently pursuing the M.S. degree with the Institute of Geophysics and Geomatics, China University of Geosciences, Wuhan, China.

His research interests include hyperspectral image processing, machine learning, and graph neural networks.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.



Yanni Dong (Senior Member, IEEE) received the B.S. degree in sciences and techniques of remote sensing from Wuhan University, Wuhan, China, in 2012, and the Ph.D. degree in photogrammetry and remote sensing from the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote sensing, Wuhan University, in 2017.

She was a Hong Kong Scholar with the Department of Land Surveying and Geo-Informatics, The Hong Kong Polytechnic University, Hong Kong. She is currently an Associate Professor with the Institute of Geophysics and Geomatics, China University of Geosciences, Wuhan. Her research interests include hyperspectral image processing, pattern recognition, and machine learning.

Dr. Dong serves as a Reviewer for more than 20 international journals, including the IEEE TRANSACTIONS ON CYBERNETICS, IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, IEEE TRANSACTIONS ON IMAGE PROCESSING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON MULTIMEDIA, IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, IEEE GEOSCIENCE AND REMOTE SENSING LETTERS, and pattern recognition (PR). She regularly serves as a PC member for International Joint Conferences on Artificial Intelligence (IJCAI) and Association for the Advancement of Artificial Intelligence (AAAI).



Yuxiang Zhang (Member, IEEE) received the B.S. degree in sciences and techniques of remote sensing and the Ph.D. degree in photogrammetry and remote sensing from Wuhan University, Wuhan, China, in 2011 and 2016, respectively.

She was a Visiting Scholar with the University of Sydney, Sydney, NSW, Australia, from 2019 to 2020. She is currently an Associate Professor with the Institute of Geophysics and Geomatics, China University of Geosciences, Wuhan. Her research interests include hyperspectral image processing, pattern recognition, and machine learning.



Hui Luo received the B.S. degree in sciences and techniques of remote sensing from Wuhan University, Wuhan, China, in 2010, and the Ph.D. degree in photogrammetry and remote sensing from the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote sensing, Wuhan University, in 2016.

She was a Visiting Scholar with The State University of New York at Buffalo, Buffalo, NY, USA, from 2014 to 2015. She is currently an Associate Professor with the School of Computer Science, China University of Geosciences, Wuhan. Her research interests include urban remote sensing, pattern recognition, and machine learning.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:12:25 UTC from IEEE Xplore. Restrictions apply.